Authx Reference - API Extra

HTTPCacheBackend

Warning

You need to install dependencies to use The HTTP Cache.

```
$ pip install authx_extra
```

class authx_extra.cache.HTTPCacheBackend

```
HTTPCacheBackend(redis, namespace=None)
```

Source code in `authx_extra/cache.py`

```
def __init__(self, redis: redis.Redis, namespace: Optional[str] = None):
    self.redis = redis
    self.namespace = namespace or HTTPCache.namespace
```

attr redis instance-attribute

```
redis = redis
```

attr namespace instance-attribute

```
namespace = namespace or namespace
```

meth set async

```
set(key, value, ttl_in_seconds=None, ttl_func=None, end_of_day=False, end_of_week=False)
```

Source code in `authx_extra/cache.py`

```python
async def set(
    self,
    key: str,
    value: str,
    ttl_in_seconds: Optional[int] = None,
    ttl_func: Optional[Callable] = None,
    end_of_day: bool = False,
    end_of_week: bool = False,
):
    ttl: int = await HTTPExpiry.get_ttl(
        ttl_in_seconds=ttl_in_seconds,
        end_of_day=end_of_day,
        end_of_week=end_of_week,
        ttl_func=ttl_func,
    )

    stringified_value = value if isinstance(value, bytes) else json.dumps(value)
    with self.redis.pipeline(transaction=True) as pipe:
        pipe.multi()
        pipe.delete(key)
        pipe.set(key, stringified_value, ex=ttl)
        log_info(msg=f"CacheSet: {key}")
        result = pipe.execute()

    del_status, set_status = result

    if del_status:
        log_info(msg=f"CacheClearedOnSet: {key}")

    if set_status:
        log_info(msg=f"CacheSet: {key}")
    return result
```

`meth` **get** `async`

```python
get(key)
```

Source code in `authx_extra/cache.py`

```python
async def get(self, key: str) -> Tuple[Union[int, None], Union[Any, None]]:
    with self.redis.pipeline(transaction=True) as pipe:
        pipe.ttl(key).get(key)
        ttl, result = pipe.execute()

    if result:
        original_val = json.loads(result)
        log_info(msg=f"CacheHit: {key}")
    else:
        original_val = None
    return ttl, original_val
```

`meth` **invalidate** `async`

```python
invalidate(key)
```

Invalidates the passed key

Source code in `authx_extra/cache.py`

```python
async def invalidate(self, key: str) -> bool:
    """Invalidates the passed key"""

    with self.redis.pipeline(transaction=True) as pipe:
        pipe.multi()
        pipe.delete(key)
        log_info(msg=f"CacheInvalidated: {key}")
        result = pipe.execute()
    return result
```

`meth` invalidate_all `async`

```python
invalidate_all(keys)
```

Invalidates a collection of keys

Source code in `authx_extra/cache.py`

```python
async def invalidate_all(self, keys: List) -> List[bool]:
    """Invalidates a collection of keys"""

    with self.redis.pipeline(transaction=True) as pipe:
        pipe.multi()
        for key in keys:
            pipe.delete(key)
            log_info(msg=f"CacheInvalidated: {key}")
        return pipe.execute()
```

March 20, 2026 March 19, 2026 GitHub